

第十章 标准标签库

- JSTL 简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- SQL 标签库
- XML 标签库（自学）



学习目标

- 了解JSTL的基本概念，下载、安装和配置
- 掌握**JSTL核心标签库**的通用标签、条件处理、循环与迭代等标签的使用
- 了解如何使用**JSTL中的SQL标签**查询、更新和删除数据库中的数据
- 了解JSTL的函数标签库和格式标签库
- 能够根据实际问题选择合适的JSTL标签完成功能



第十章 标准标签库

- JSTL 简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- SQL 标签库
- XML 标签库（自学）

JSTL简介

- JSTL 全名为 **JavaServer Pages Standard Tag Library**，是由 JCP (Java Community Process) 所指定的标准标签库，提供Web 开发的通用功能。
- 利用JSTL与EL配合，可简化JSP页面的开发。



利用taglib指令引入标签库

- `<%@taglib prefix=“缩略名” uri=“标签的位置” %>`
 - prefix指明在JSP页面中引用标签时所使用的
前缀
 - uri告知容器如何找到标签描述文件和标签库
- 作用：将标记引入到JSP页面中，供在需要的地方使用。

JSTL简介

JSTL	前置 名称	URI	范例
核心 标签库	c	http://java.sun.com/jsp/jstl/core	<c:out>
格式 标签库	fmt	http://java.sun.com/jsp/jstl/fmt	<fmt:formatDate>
SQL 标签库	sql	http://java.sun.com/jsp/jstl/sql	<sql:query>
XML 标签库	x	http://java.sun.com/jsp/jstl/xml	<x:forEach>
函数 标签库	fn	http://java.sun.com/jsp/jstl/functions	<fn:split>



JSTL的安装和配置

- JSTL主要由Apache组织的Jakarta Project所实现，因此需要到
<http://archive.apache.org/dist/jakarta/taglibs/standard/binaries/jakarta-taglibs-standard-1.1.2.zip>下载，将jstl.jar和standard.jar复制到web应用WEB-INF/lib目录下，然后就可以在JSP页面中使用JSTL了。
- 注意：JSTL1.1以上的版本必须在支持 Servlet 2.4 且 JSP 2.0 以上版本的 Container 才可使用。

测试JSTL是否安装成功

firstJSTL.jsp

```
<%@ page contentType="text/html; charset=GBK" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<head>
<title>测试你的第一个使用JSTL的网页</title>
</head>
<body>
<c:out value="欢迎测试你的第一个使用JSTL的网页"/>
</br>你使用的浏览器是: </br>
<c:out value="$ {header ['User-Agent']}" />
<c:set var="a" value="David O'Davies" />
<c:out value="David O'Davies" escapeXml="true"/>
</body>
</html>
```

<http://localhost:8080/ch12/firstJSTL.jsp>



第十章 标准标签库

- JSTL 简介
- **核心标签库**
- 格式标签库（自学）
- 函数标签库（自学）
- SQL 标签库
- XML 标签库（自学）

核心标签库 (Core tag library)

分类	功能分类	标签名称
Core	通用标签	out、set、remove、catch
	条件标签	if、choose、when、otherwise
	循环迭代标签	forEach、forTokens
	URL标签	import、param url、param redirect、param

通用标签

- 包含四个标签：<c:out>、<c:set>、<c:remove>和<c:catch>
 - <c:out>主要用来显示数据的内容
 - <c:set>用来将变量储存至JSP范围中或JavaBean 属性中。
 - <c:remove>用来[在指定的范围内]移除变量。
 - <c:catch>主要用来捕获标签体内的语句执行时所抛出的异常。



<c:out>

- <c:out>主要用来显示数据的内容

- 语法:

- 语法 1: 没有本体 (body) 内容

`<c:out value="value" [escapeXml="{true/false}"] [default="默认值"] />`

- 语法 2: 有本体内容

`<c:out value="value" escapeXml="{true/false}"/>`

默认值

`</c:out>`

名称	说明	EL支持	类型	必须	默认值
value	需要显示出来的值	Y	Object	是	无
default	如果 value 的值为 null, 则显示 default 的值	Y	Object	否	无
escapeXml	是否转换特殊字符, 如: <转换成<	Y	boolean	否	true

- 例子



<c:set>

- <c:set>用来将变量储存至JSP范围中或 JavaBean 属性中。
- 四种语法
 - 语法 1: 将value的值储存至scope范围的varName变量中

```
<c:set value="value" var="varName"
scope="{ page/request/session/application }"/>
```
 - 语法 2: 将本体内内容储存至scope范围的varName变量中

```
<c:set var="varName" [scope="{ page/request/session/application }"]>
... 本体内内容
</c:set>
```
 - 语法 3: 将value的值存至target对象的属性中

```
< c:set value="value" target= "${target}" property="propertyName" />
```
 - 语法 4: 将本体内内容存至target对象的属性中

```
<c:set target="target" property="propertyName">
... 本体内内容
</c:set>
```

Target为JavaBeans或Map类型

例子:

<c:set>的属性

名称	说明	EL	类型	必须	默认值
value	要被储存的值	Y	Object	否	无
var	欲存入的变量名称	N	String	否	无
scope	var 变量的 JSP 范围	N	String	否	page
target	为一 JavaBean 或 java.util.Map 对象	Y	Object	否	无
property	指定 target 对象的属性	Y	String	否	无



<c:remove>

- <c:remove>用来[在指定的范围内]移除变量。
- 语法

```
<c:remove var="varName"  
[scope="{ page/request/session/application }"] />
```

名称	说明	EL	类型	必须	默认值
var	欲移除的变量名称	N	String	是	无
scope	var 变量的 JSP 范围	N	String	否	page

<c:catch>

- <c:catch>主要用来捕获标签体内的语句执行时所抛出的异常。

- 语法

<c:catch [var="varName"] >

... 欲抓取错误的部分

</c:catch>

属性	说明	EL	类型	必须	默认值
var	用来储存错误信息的变量	N	String	否	无



条件标签

- 包含四个标签：<c:if>、<c:choose>、<c:when>和<c:otherwise>
 - <c:if>用于条件判断
 - <c:choose><c:when><c:otherwise>联合使用，用于多分支条件选择。

<c:if>

- <c:if>用于条件判断

- 语法 1：没有本体内容 (body)

```
<c:if test="testCondition" var="varName"  
[scope="{page|request|session|application}"]/>
```

- 语法 2：有本体内容

```
<c:if test="testCondition" [var="varName"]  
[scope="{page|request|session|application}"]>
```

本体内容

```
</c:if>
```

名称	说明	EL	类型	必须	默认值
test	若表达式结果为 true，执行本体内容，否则相反	Y	boolean	是	无
var	用来储存 test 运算后的结果，即 true 或 false	N	String	否	无
scope	var 变量的 JSP 范围	N	String	否	page

- 例子



<c:choose><c:when><c:otherwise>

- 联合使用，用于多分支条件选择。
- 语法
 - <c:choose>
 - <c:when test= “逻辑表达式1” >内容1</c:when>
 - <c:when test= “逻辑表达式2” >内容2</c:when>
 - ...
 - <c:when test= “逻辑表达式n” >内容n</c:when>
 - <c:otherwise> 内容</c:otherwise>
 - </c:choose>
- <c:when>必须有 test 属性，当 test 中的表达式结果为 true 时，则会执行本体内内容；如果为false 时，则不会执行。
- 例子



循环迭代标签

- 主要包含2个标签：<c:forEach>和<c:forEachTokens>。
 - <c:forEach> 可以实现固定次数的循环，也可以对集合元素进行迭代。
 - <c:forEachTokens> 用来浏览一字符串中所有的成员，其成员是由定义符号(delimiters)所分隔的。



<c:forEach>

- <c:forEach> 可以实现固定次数的循环，也可以对集合元素进行迭代。

- 语法

- 语法 1：对集合对象迭代

```
<c:forEach [var="varName"] items="collection"  
           [varStatus="varStatusName"] [begin="begin"]  
           [end="end"] [step="step"]>
```

本体内内容

```
< /c:forEach>
```

- 语法 2：迭代固定的次数

```
<c:forEach [var="varName"] [varStatus="varStatusName"]  
           begin="begin" end="end" [step="step"]>
```

本体内内容

```
</c:forEach>
```

- 例子：

<c:forEach>的属性

名称	说明	EL	类型	必须	默认值
var	当前元素	N	String	否	无
items	用于迭代的集合对象	Y	Arrays Collection Iterator Enumeration Map String	否	无
varStatus	当前元素的状态	N	String	否	无
begin	开始的位置	Y	int	否	0
end	结束的位置	Y	int	否	最后一个成员
step	每次迭代的步长	Y	int	否	1

- 如果要循序浏览一个集合对象，并将内容显示出来，就必须有 **items** 属性。
- 没有指定**items**属性，实现固定次数的循环。
- **varStatus**属性包含index、count、first和last4个值，表示当前元素的索引、元素总数、是否为第一个元素、是否为最后一个元素



<c:forTokens>

- <c:forTokens> 用来浏览一字符串中所有的成员，其成员是由定义符号 (delimiters) 所分隔的。

- 语法

```
<c:forTokens items="stringOfTokens"
  delims="delimiters" [var="varName"]
[varStatus="varStatusName"] [begin="begin"]
[end="end"] [step="step"]>
  本体内内容
</c:forTokens>
```

- 例子

<c:forTokens>的属性

名称	说明	EL	类型	必须	默认值
var	当前元素	N	String	否	无
items	被迭代的字符串	Y	String	是	无
delims	分割符	N	String	是	无
varStatus	存放当前元素的信息	N	String	否	无
begin	开始的位置	Y	int	否	0
end	结束的位置	Y	int	否	最后一个成员
step	每次迭代的间隔数	Y	int	否	1

- 说明: items 的内容必须为字符串; 而 delims 是用来分割 items 中定义的字符串之字符。



URL 标签

- `<c:import>`实现资源导入
- `<c:url>`格式化URL
- `<c:redirect>`重定向
- `<c:param>`实现参数传递
- 前面三个标签都可以把`<c:param>`作为子标签。

<c:import>

- <c:import> 用于在JSP页面中导入资源。

- 语法 1:

```
<c:import url="url" [context="context"] [var="varName"]  
[scope="{page|request|session|application}"]  
[charEncoding="charEncoding"]>
```

本体内内容

```
</c:import>
```

- 语法 2:

```
<c:import url="url" [context="context"]  
varReader="varReaderName"  
[charEncoding="charEncoding"]>
```

本体内内容

```
</c:import>
```

- 例子

<c:import>的属性

名称	说明	EL	类型	必须	默认值
url	一导入资源的URL地址	Y	String	是	无
context	/ 加上本地Web应用的名称, 当url的值为相对地址时使用	Y	String	否	无
var	保存导入内容的变量	N	String	否	无
scope	var 变量的 JSP 范围	N	String	否	Page
charEncoding	导入资源时使用的编码字符集	Y	String	否	无
varReader	保存导入内容的java.io. Reader 对象	N	String	否	无

• 说明

- <c:import>中必须要有 url 属性, 用来设定被包含网页的地址, 它可以为绝对地址或是相对地址。
- 如果要包含在同一个服务器上, 但并非同一个 web 站点的文件时, 就必须加上 context 属性。
- 可以在<c:import>的本体内容中使用<c:param>, 它的功用主要是: 可以将参数传递给被包含的文件, 它有两个属性 name 和 value



<c:url>

- <c:url>主要格式化URL，格式化后的结果可以输出或保存到变量中。

- 语法

- 语法 1：没有本体内容

```
<c:url value="value" [context="context"] [var="varName"]  
[scope="{page|request|session|application}"] />
```

- 语法 2：本体内容代表查询字符串 (Query String)
参数

```
<c:url value="value" [context="context"] [var="varName"]  
[scope="{page|request|session|application}"] >
```

若干<c:param> 标签

```
</c:url>
```

- 例子：

<c:url>的属性

名称	说明	EL	类型	必须	默认值
value	用于输出或格式化的 URL	Y	String	是	无
context	/ 加上本地Web应用的名称，当url的值为相对地址时使用	Y	String	否	无
var	保存格式化后URL的变量	N	String	否	无
scope	var 变量的 JSP 范围	N	String	否	Page

<c:redirect>

- <c:redirect>用于将客户请求重新定向到一个新URL。

- 语法 1：没有本体内容

```
<c:redirect url="url" [context="context"] />
```

- 语法 2：本体内容代表查询字符串 (Query String) 参数

```
<c:redirect url="url" [context="context"] >
```

```
<c:param>
```

```
</c:redirect >
```

名称	说明	EL	类型	必须	默认值
url	导向的目标地址	Y	String	是	无
context	/ 加上本地Web应用的名称，当url的值为相对地址时使用	Y	String	否	无

- 例子

- 项目1-使用JSTL实现图书管理系统的视图层
 - 第9章中基于Model 2的图书管理系统的视图层文件共5个，其中list.jsp和edit.jsp文件需要用JSTL改写，以达到在页面实现中不出现任何Java脚本。
 - 基本思想是使用EL表达式和JSTL中的<c:forEach>标签实现对对象属性的访问以及循环迭代操作。



第十章 标准标签库

- JSTL 简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- SQL 标签库
- XML 标签库（自学）

格式化标签库（自学）

格式化标签库	国际化	setLocale requestEncoding
	消息	bundle message param setBundle
	数字、日期格式化	formatNumber formatDate parseDate ParseNumber setTimeZone timeZone

国际化标签

- `<fmt:setLocale>` 设定语言区域
- `<fmt:requestEncoding>` 设定请求字符串编码



<fmt:setLocale>

- 用来设置用户的语言地区

- 语法：

- <fmt:setLocale value =“ { ll | ll-CC } ”
[variant=“variant”]
[scope=“ {page|request|session|application} ”]/>

- 说明：

- ll：为两个字母代表语言代码，例如：zh(中文)
CC：代表国家代码，例如：CN(中国)
 - 其中value的取值为地区代码，具体的取值参见
<http://www.w3.org/WAI/ER/IG/ert/iso639.htm>



- 默认情况下，l18n标记依据浏览器的设定来确定本地属性值。使用<fmt:setLocale>标记会覆盖浏览器本地属性设置。
- 在Web应用的web.xml文件中，可以使用<context-param>标记设置整个Web应用默认的本地属性。

属性说明

名称	说明	E L	类型	必须	默认值
value	地区代码，其中最少要有两个字母的语言代码，如zh、en，然后也可以再加上两个字母的国家代码和地区代码，如：US，CN两者可以由“—”或“_”相联结起来，例如zh_CN	Y	String/ java.util.Locale	Y	无
variant	供货商或浏览器的规格，如WIN代码Windows，Mac代码Macintosh	N	String	N	无
scope	地区设置的适用范围	N	String	N	page

注：当value值为null时，则使用默认的区域设置



<fmt:requestEncoding>

- <fmt:requestEncoding>用来设置字符串的编码
- 语法：
 - <fmt:requestEncoding [value="charsetName"]/>
 - 说明：该标记只有一个value属性，如果没有设定value属性，则它会自动去寻找合适的编码方式
- 默认的编码格式：ISO-8859-1
- 对于非ISO-8859-1的字符（如：中文），需要显式指定字符集编码。
- 例如：<fmt:requestEncoding value="GBK"/>



消息标记库

- 消息标记库中包含四个标记：
`<fmt:message>``<fmt:param>``<fmt:bundle>``<fmt:setBundle>`。
- 主要作用为获取系统设定的语言资源，使用这些标记可以轻易地使Web应用支持国际化。



资源文件

- 资源包：为了能够在浏览器端用本地属性指定的语言显示内容，一般需要开发人员为需要支持的每个本地属性都提供一个资源集合，通常称为资源包。
- 资源包中包含标准的“键-值”对表示的本地属性。
- 资源包可以用java类实现，也可以用一个文本的属性文件（资源文件）实现：
 - 一种方式是使用java类实现的资源包
 - 一种是使用纯文本实现的资源文件
- 资源包的命名：文件名中只有后缀的本地属性串不同。
- 资源包的内容：文件内容使用相同的键名，不同语言的键值。



<fmt:bundle>

- <fmt:bundle>主要用来设定本体内容的数据源
- 语法:

```
<fmt:bundle basename="basename"  
    [prefix="prefix"]>
```

本体内容

```
</fmt:bundle>
```

注：如果basename为null、空或找不到资源文件时，在网页上会产生” ???<key>???”的错误信息

<fmt:bundle>的属性

名称	说明	EL	类型	必须	默认值
basename	要是用资源文件的名称	Y	String	Y	无
prefix	前置关键字	Y	String	N	无

- 说明：basename属性设置要使用资源的名称，如如果资源文件的名称为MyResource.properties，那么basename的值为MyResource。注意千万不可以有任何文件类型。

<fmt:setBundle>

- <fmt:setBundle>可以用来设定默认的资源文件，或者将其设置到指定的属性范围中去。
- 语法：
 - <fmt:setBundle basename="basename"
[var="varName"]
[scope="{page|request|session|application}"]/>
 - 注意：若basename为null、空或者无法找到资源文件时，在网页上会产生”???<key>???”错误信息



名称	说明	EL	类型	必须	默认值
basename	要是用资源文件的名称	Y	String	Y	无
var	存储资源的名称	N	String	N	无
scope	var变量的JSP范围	N	String	N	page

Baseline设定要使用的资源文件，和<fmt:bundle>相同，如果没有设定**var**时，那么设定好的资源文件会成为默认的资源文件，让在同一个网页或同一个属性范围内<fmt:message>都可以直接使用此资源文件。相反，如果设定**var**时，那么将会将资源文件存储到**varName**中，当使用<fmt:message>时，必须使用**bundle**这个属性来指定。如：

```
<fmt:setBundel basename="Myresource" scope="session"
var="myResource"/>
```

```
<fmt:message key="str" bundle="${myResource}">
```

如果没有指定**var**，则只需写成：

```
<fmt:setBundel basename="MyResource" scope="session"/>
```

```
<fmt:message key="str"/>
```

<fmt:message>

- <fmt:message>会从指定的资源中把指定的关键字获取出来
- 语法：
 - 语法1：
 - <fmt:message key="messagekey"
[bundle="resourceBundle"]
[var="varname"] [scope="page|request|session|application"]/>

<fmt:message>

- 语法2:

```
- <fmt:message key="messagekey"  
  [bundle="resourceBundle"]  
  [var="varname"] [scope="page|request|session|application"] >  
  <fmt:param />  
</fmt:message>
```

- 语法3:

```
- <fmt:message [bundle="resourceBundle"]  
  [var="varname"] [scope="page|request|session|application"] >  
  索引  
  <fmt:param />  
</fmt:message>
```



<fmt:message>

名称	说明	EL	类型	必须	默认值
key	索引	y	String	否	无
bundle	使用的数据来源	Y	LocalizationContext	否	无
var	用来存储国际化信息	N	String	否	无
scope	var变量的JSP范围	N	String	否	page

注：若key为null或空是，在网页上产生“????”的错误信息，若找不到资源文件，在网页上灰产生”???<key>???“的错误信息
说明：

如果<fmt:message>没有key属性，那么<fmt:message>将会自动从本体内容中寻找关键字，再从关键字中寻找对应的结果，显示在页面中。
Bundle属性则是指定使用的资源文件，var和scope用来存储要显示的信息。如果有var属性，该标记则不会把结果显示在页面中，而是将结果存储在varname中，要进行显示则必须使用<c:out>或\${}EL表达式



<fmt:param>

- 当我们从资源文件获取内容时，有时候需要动态设置参数，<fmt:param>就是用来提供这些参数
- 语法1：通过value属性设置参数值
 - <fmt:param value="messageParameter"/>
- 语法2：通过本体设定参数值

```
<fmt:param>
    本体内容
</fmt:param>
```
- 说明：value如果没有设定要给予的参数，那么就会默认去获取本体内容当作要传递的参数



- `prefix`用来设置前置关键字，例如当`properties`文件的内容如下时：
 `Requestinfo.label.method=Method`
 `Requestinfo.label.protocol=Protocol`
- 我们就可以使用如下的方式使其内容显示出来：
 `<fmt:bundle basename="MyResource"`
 `prefix="requestinfo.label.">`
 `<fmt:message key="method"/>`
 `<fmt:message key="protocol"/>`
 `</fmt:bundle>`



数字、日期格式化

- 共包含6个标签，分别为
`<fmt:formatNumber>``<fmt:parseNumber>``<fmt:formatDate>``<fmt:parseDate>``<fmt:setTimeZone>``<fmt:timeZone`
- 分别用来解析或格式化数字、日期以及货币等。一般用于将数字、日期等转换成指定地区或自定义的显示格式。

<fmt:timeZone>

- 语法：

<fmt:timeZone value="timeZone">

 本体内内容，如：格式化的时间，日期等

</fmt:timeZone>

- 作用：指定时区，其本体内内容会以指定时区的时间格式来解析和显示。由于该标记的设置只影响其本体内内容，也称暂时时区。
- timeZone是时区的ID，例如：America/Los_Angeles, GMT-8等。
- 如果value的值为空或null，则默认为GMT时区。



<fmt:setTimeZone>

- 语法:

```
<fmt:setTimeZone value="timeZone" [var="varName"]  
[scope="page|request...."]/>
```

- 作用: 将value属性指定的时区值存储到一个var变量中。变量的有效范围由scope指定, 默认页内有效。
- 如果value的值为空或者null, 则默认为GMT时区。
- 如果没有使用var指明变量, 则value的值保存在当前配置变量 (javax.servlet.jsp.jstl.fmt.timeZone) 中, 作为有效范围内的默认时区。



<fmt:formatNumber>

- <fmt:formatNumber>会依据设定的区域将数字改为适当的格式。
- 语法

- 语法1：没有本体内容

- <fmt:formatNumber value="numericValue"
[type="{number|currency|percent}"]
[pattern="customPattern"]
[currencyCode="currencyCode"]
[currencySymbol="currencySymbol"]
[groupingUsed="{true|false}"]
[maxIntegerDigits="maxIntegerDigits"]
[minIntegerDigits="minIntegerDigits"]
[maxFractionDigits="maxFractionDigits"]
[minFractionDigits="minFractionDigits"]
[var="varName"]
[scope="{page|request|session|application}"]

/>

<fmt:formatNumber>

- 语法2：本体为欲格式化的内容

- <fmt:formatNumber
[type="{number|currency|percent}"]
[pattern="customPattern"]
[currencyCode="currencyCode"]
[currencySymbol="currencySymbol"]
[groupingUsed="{true|false}"]
[maxIntegerDigits="maxIntegerDigits"]
[minIntegerDigits="minIntegerDigits"]
[maxFractionDigits="maxFractionDigits"]
[minFractionDigits="minFractionDigits"]
[var="varName"]
[scope="{page|request|session|application}"]>

欲格式化的数字

</fmt:formatNumber>

<fmt:formatNumber>的属性

名称	说明	EL	类型	必须	默认值
value	欲格式化的数字	Y	String / Number	否	无
type	指定单位（数字、当地货币、百分比）	Y	String	否	number
patten	格式化数字的样式	Y	String	否	无
currencyCode	ISO-4217 码	Y	String	否	无
currencySymbol	货币符号，如¥，\$	Y	String	否	无
groupingUsed	是否使用区隔数字，如： 123, 456, 789	y	boolean	否	true
maxIntegerDigits	整数部分最多显示多少位	Y	int	否	无
minIntegerDigits	整数部分最少显示多少位	Y	int	否	无
maxFractionDigits	小数点后最多显示多少位	Y	int	否	无
minFractionDigits	小数点后最少显示多少位	Y	int	否	无
var	存储已格式之数字	N	String	否	无
scope	var 变量的 JSP 范围	N	String	否	page



关于<fmt:formatNumber>的说明

- 当无法确定区域时，则输出的格式为
`Number.toString()`
- 如果pattern为null或者空时，则pattern将会被忽略
- 详细的pattern设定可以参考
<http://java.sun.com/j2se/1.4.2/docs/api/java/text/DecimalFormat.html>
- currencyCode为货币代码，如美元就是USD，详细的
货币代码可以至http://www.bsi-global.com/technical+information/Publications/_Publications/tig90x.doc

<fmt:parseNumber>

- 将字符串类型的数字、货币或百分比转换为数字
- 语法：

- 语法1：没有本体的内容

```
<fmt:parseNumber value="numericValue"
```

```
  [type="{number|currency|percent}"]
```

```
  [pattern="customPattern"]
```

```
  [parseLocale="parseLocale"]
```

```
  [integerOnly="{true|false}"]
```

```
  [var="varName"]
```

```
  [scope="{page|request|session|application}"]/>
```



- 语法2：有本体的内容

```
<fmt:parseNumber  
  [type="{number|currency|percent}"]  
    [pattern="customPattern"]  
    [parseLocale="parseLocale"]  
    [integerOnly="{true|false}"]  
    [var="varName"]  
    [scope="{page|request|session|application}"]>  
待转换为字符串的字符串  
</fmt:parseNumber>
```

<fmt:parseNumbe>的属性

名称	说明	EL	类型	必须	默认值
value	待格式化的数字	Y	String /Number	N	无
type	指定欲格式属性的类型，分别为 (数字、当地字符以及百分比)	Y	String	N	number
pattern	格式化数字的样式	Y	String	N	无
parseLocale	用来替代默认的地区设置	Y	String /java.util.Locale	否	无
integerOnly	是否只显示整数部分	Y	boolean	否	false
var	存储已格式化的数字	N	String	否	无
scope	var的JSP范围	N	String	否	page

<fmt:formatDate>

- 作用：以指定的时区格式化显示日期对象（`java.util.Date`）。
- 语法：

```
<fmt:formatDate value="date"  
[type="{time|date|both}"]  
[dateStyle="{default|short|medium|long|full}"]  
[timeStyle="{default|short|medium|long|full}"]  
[pattern="customPattern"]  
[timeZone="timeZone"]  
[var="varName"]  
[scope="{page|request|requestAttributes|session|application}"]/>
```

- `value`指定需要格式化显示的日期和时间。
- `type`指定给定的数据的处理方式日期、时间还是日期时间都处理。
- `dateStyle`和`timeStyle`指定日期和时间的显示格式。
- `Pattern`属性指定自定义格式，如“dd/MM/yyyy”等。见P197
- `timeZone`指定时区，如果没有指定，默认使用本地属性中的时区。
- 示例：`formatDate1.jsp`，`fomatDate2.jsp`

<fmt:parseDate>

- 作用：将字符串表示的日期和时间解析成日期对象（java.util.Date）。

- 语法：

```
<fmt:parseDate value="dateString"  
[type="{time|date|both}"]  
[dateStyle="{default|short|medium|long|full}"]  
[timeStyle="{default|short|medium|long|full}"]  
[patter="customPattern"]  
[timeZone="timeZone"] [parseLocale="parseLocale"]  
[var="varName"]  
[scope="{page|request|request....}"]/>
```



- `value`属性指定需要解析成日期对象的字符串。该属性可以省略，由本体来指定要解析的内容。
- 解析结果可以使用属性`var`指定的变量存储。变量的有效范围由`scope`指定。如果没有`var`属性，解析结果会输出到JSP页面，否则不会输出。
- 属性`parseLocale`指定本地属性的值。类型为`java.util.Locale`。



第十章 标准标签库

- JSTL 简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- SQL 标签库
- XML 标签库（自学）



函数标签库（自学）

- 在JSP页面中引入函数标签库需要使用如下指令：

```
<%@ taglib prefix="fn"  
uri="http://java.sun.com/jstl/functions" %>
```

- 在EL表达式中使用标准函数采用如下格式：
\${fn:函数名(参数列表)}

	函数 齐鲁工业大学 QILU UNIVERSITY OF TECHNOLOGY	Web 应用开发 说明	eclipse 	
contains(s1,s2)	判断字符串 s2 是否在字符串 s1 中			
containsIgnoreCase(s1,s2)	同上，但判断时忽略大小写			
startsWith(s1,s2)	判断字符串 s1 是否以字符串 s2 开头			
endsWith(s1,s2)	判断字符串 s1 是否以字符串 s2 结尾			
escapeXml(s)	忽略字符串 s 中的 XML 标签			
indexOf(s1,s2)	返回字符串 s2 在字符串 s1 中第一次出现的位置			
split(s1,s2)	用 s2 指定的分割字符集分割字符串 s1，返回值为字符串数组。			
join(String[] s1,s2)	将字符串数组 s1 中的所有字符串，以字符串 s2 中指定的连接符组合。			
replace(s1,s2,s3)	将字符串 s1 中的子串 s2 用字符串 s3 代替			
trim(s)	去除字符串 s 两端空格			
substring(s,i1,i2)	返回字符串 s 中从 i1 开始到 i2-1 结束的子串			
substringAfter(s1,s2)	抽取字符串 s1 中子串 s2 之后的字符串			
substringBefore(s1,s2)	抽取字符串 s1 中子串 s2 之前的字符串			
toLowerCase(s)	转换字符串 s 中的所有字符为小写字符			
toUpperCase(s)	转换字符串 s 中的所有字符为大写字符			
length(s)	返回字符串 s 的长度			

第十章 标准标签库

- JSTL 简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- **SQL 标签库**
- XML 标签库（自学）



SQL标记库

- JSTL中的SQL标记库提供了与数据库交换的能力。
- 要使用SQL标记库，必须先使用taglib指令导入，语法格式如下：

```
<%@taglib prefix="sql"  
uri="http://java.sun.com/jsp/jstl/sql"  
%>
```

指出：数据库交互属于业务逻辑层，所以数据库标签库违背了mvc框架。仅仅了解即可。



使用SQL标记访问数据库的步骤

- 使用SQL标记访问数据库的主要步骤如下：

1. 指定数据源

使用<sql:setDataSource>标记

2. 进行查询或更新操作

使用<sql:query>标记及<sql:update>标记进行增删改查，<sql:param>和<sql:dateParam>这两个标记用来动态地设定sql参数

<sql:transaction>标记用来处理事务

3. 对返回的结果进行处理

<c:forEach>



SQL标记访问数据库步骤1——指定数据源

- 使用`<sql:setDataSource>`标记来指定数据源，其语法为：

```
<sql:setDataSource url= "jdbcUrl"  
    [driver= "driverClassName" ] [user= "userName" ]  
    [password= "password" ] [var= "varName" ]  
    [scope= "{page|request|session|application}" ] />
```

- `url`属性用于指定连接的url。如连接Oracle数据库时的url为：`jdbc:oracle:thin:@[ip]:[port]:[sid]`
- `driver`属性用于指定驱动程序，如Oracle数据库的驱动程序为：`oracle.jdbc.driver.OracleDriver`
- `user`属性和`password`属性则分别用于指定用户名和密码。
- `scope`属性指定该设置的有效范围

(注意：如果使用了`var`属性，则只是将该数据源保存到一个变量中)

SQL标记访问数据库步骤2——进行查询或更新操作

- 使用`<sql:query>`标记可以进行数据库查询，其语法为：

```
<sql:query sql=“查询语句” var=“varName”  
[dataSource=“dataSource” ]  
[scope=“{page|request|session|application}” ]  
[maxRows=“rowMax” ]  
[startRow=“startRow” ]  
/>
```

- 也可以将查询语句放在标记本体中，如：

```
- <sql:query var=“varName” [dataSource=“dataSource” ]  
[scope=“{page|request|session|application}” ]  
[maxRows=“rowMax” ]  
[startRow=“startRow” ]  
  查询语句  
- </sql:query>
```

SQL标记访问数据库步骤3——处理返回的结果

- sql属性指定需要执行的SQL查询语句；属性var指定存储查询结果对象的变量名；scope属性指定有效范围。
- 如果前面执行的是查询操作，则会将返回的结果保存在一个Result类型的变量中（返回的结果一般是一个二维表）
- 该变量一般有如下几个属性：
 - rows：一个Map对象的数组
 - rowsByIndex：一个对象数组的数组
 - columnNames：列名的数组
 - rowCount：返回结果的行数
 - limitedByMaxRows：查询是否有最大行数限制--boolean型

- <sql:query>查询的结果一般是一个二维表，假如保存在变量x中，并且该表中有三个字段name、sex、score。则输出该表信息的方法为：

(1). 使用rows属性

```
<c:forEach items= "${x.rows}" var= "temp" >  
    ${temp.name}、${temp.sex}、${temp.score} <br>  
</c:forEach>
```

(2). 使用rowsByIndex属性

```
<c:forEach items= "${x.rowsByIndex}"  
var= "temp" >  
    ${temp[0]}、 ${temp[1]} 、 ${temp[2]} <br>  
</c:forEach>
```

- 另两个属性的使用方法如下：

(1). 可以使用columnNames属性输出各列的名称

```
<c:forEach items= “${x.columnNames}”  
var= “temp” >  
    ${temp}  
</c:forEach>
```

(2). 可以使用rowCount属性输出返回结果的行数

```
    ${x.rowCount}
```

- 使用`<sql:update>`进行数据库的更新操作，如创建表、插入和删除记录等。

其常用语法为：

- `<sql:update sql= “更新语句” [dataSource= “dataSource” ]`
 `[var= “varName” ]`
 `[scope= “{page|request|session|application}” ]`
 - `/>`
- 也可以将更新语句放到标记本体中
 - `<sql:update [dataSource= “dataSource” ]`
 `[var= “varName” ]`
 `[scope= “{page|request|session|application}” ]>`
 更新语句
 `</sql:update>`
- 可选var属性用于指定修改操作影响的数据记录行数。

- `<sql:param>`和`<sql:dateParam>`这两个标记用来动态地设定sql参数
 - `<sql:dateParam>`只是用来设置日期相关的参数，如
timeStamp、date、time

- `<sql:transaction>` 标记用来处理事务。
- 语法：
 - `<sql:transaction [dataSource=“dataSource”]
isolation=isolationLevel>`
 多个`<sql:query>`和`<sql:update>`标记
 `</sql:transaction>`



- 项目2-SQL标签库的使用
- 使用SQL标签库实现对数据库ch09中user表数据的显示、修改和删除功能。
- 编写思路
 - (1) 使用<sql:setDataSource>标签设置数据源，并将设置的数据源存储在session对象中。
 - (2) 使用<sql:query>标签完成数据查询功能，查询的结果使用<c:forEach>标签进行遍历显示。
 - (3) <sql:update>标签完成数据修改和删除功能。数据修改前，需要用<sql:query>标签执行单条记录的查询操作，查询后的内容显示在form表单中。
- 详见教材P278-282



第十章 标准标签库

- JSTL简介
- 核心标签库
- 格式标签库（自学）
- 函数标签库（自学）
- SQL标签库
- XML标签库（自学）



XML标签库（自学）

- JSTL的XML标签库提供了解析XML文档、提取XML文档中数据、流程控制和XSLT转换等功能。按用途分类：
 - XML核心标签，包括：`<x:parse>`、`<x:out>`和`<x:set>`。
 - XML流程控制标签，包括：`<x:if>`、`<x:choose>`、`<x:when>`、`<x:otherwise>`和`<x:forEach>`。
 - XML转换标签，包括：`<x:transform>`和`<x:param>`。



小结

- JSTL的出现即避免了页面中Java脚本的使用，又免去了自定义标签中功能的重复定义。
- JSTL包含5类标准标签库：**核心标签库**，**格式标签库**，**函数标签库**，**SQL标签库**和**XML标签库**。它们实现了内容输出，循环迭代，国际化显示，日期数字格式化，字符串处理，数据库访问及XML文档解析等主要功能。
- JSTL和EL的结合使用可以使得JSP页面中不再出现任何Java脚本，**简化了JSP页面的开发**，使得页面清晰简洁，便于理解和维护。

作业

- 1、什么是JSTL？
- 2、JSTL的类别及每类的功能。
- 3、JSTL中用于实现多个条件选择的标签？
- 4、JSTL中用于建立与数据库连接的标签？